MODULE 4 · NETWORKING

TLS Ingress

Terminating HTTPS at the edge — automatically, with cert-manager

WHY IT MATTERS

Plaintext HTTP at the edge doesn't survive contact with reality

An Ingress without TLS serves **cleartext HTTP** — credentials, cookies, and payloads travel in the open.

cert-manager automates the certificate lifecycle: issue, store in a Secret, renew before expiry — no manual OpenSSL steps per Ingress.

BY THE END YOU CAN

- Explain Issuer → Certificate → Secret
- Wire `tls:` into an Ingress
- Verify a Certificate is `Ready`
- Read SANs on a self-signed cert

CONCEPT

A Certificate is a request; cert-manager fulfils it into a Secret

An **Issuer** / **ClusterIssuer** is a certificate authority cert-manager knows how to talk to — here, a **SelfSigned** one that needs no external validation.

A **Certificate** object declares what you want (which DNS names, which Secret to write). cert-manager watches it, asks the issuer, and writes the result into a `kubernetes.io/tls` Secret. The Ingress just points `tls.secretName` at that Secret.

ISSUER TYPES

SelfSigned is one of several backends

ISSUER TYPE	TRUST	USE CASE
`SelfSigned`	None — the cert signs itself	Local dev / training, internal-only TLS
`CA`	An internal CA key pair you provide	Private/internal PKI, mTLS
`ACME`	A public CA (e.g. Let's Encrypt)	Publicly trusted, browser-trusted certs

ISSUER VS CLUSTERISSUER

An `Issuer` is namespaced; a `ClusterIssuer` is cluster-wide and can be referenced from any namespace — what this lab uses.

MANIFEST ANATOMY

A Certificate names its Secret and its issuer

```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: training-tls-cert
  namespace: training
spec:
  secretName: training-tls-secret  # Secret to write
  issuerRef:
    name: selfsigned-issuer
    kind: ClusterIssuer
  dnsNames:
    - training-tls.local      # becomes the SAN
  duration: 2160h
  renewBefore: 360h
```

- **secretName** — where the issued `tls.crt` / `tls.key` land
- **issuerRef** — which Issuer/ClusterIssuer signs it
- **dnsNames** — the hostnames the cert is valid for (its SANs)
- **duration / renewBefore** — validity window and how early cert-manager renews it

WATCH

No `commonName` here — this cert has an **empty Subject**. Identity comes entirely from `dnsNames`.

DO IT — TWO WAYS

Scaffold the shape, hand-add the TLS block

```
$ kubectl create ingress tls-ingress \
    --rule="training-tls.local/*=tls-web-svc:80" \
    --dry-run=client -o yaml > ingress-tls.yaml
```

The imperative form only scaffolds `rules:` — the `tls:` block and cert-manager objects are always hand-written or templated. Full result after adding it:

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-ingress
  namespace: training
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - training-tls.local
      secretName: training-tls-secret
  rules:
    - host: training-tls.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: tls-web-svc
                port:
                  number: 80
```

Same object as `labs/4.6/ingress-tls.yaml` — apply with `kubectl apply -f ingress-tls.yaml`.

SEE IT

Ready, typed, and terminating HTTPS

Three checks confirm the chain worked: the `Certificate` is `Ready`, the `Secret` is type `kubernetes.io/tls`, and the `Ingress` lists the host under its TLS section.

Use `kubectl wait --for=condition=Ready certificate/...` before moving on — cert-manager issuance is asynchronous.

```
$ kubectl get certificate training-tls-cert -n
training
NAME                READY   SECRET
training-tls-cert   True    training-tls-secret

$ kubectl get secret training-tls-secret -n training
NAME                  TYPE                DATA
training-tls-secret   kubernetes.io/tls   2
```

CHECKING THE CERTIFICATE

Self-signed means an empty Subject — trust the SAN

Decode the Secret and read the cert: with no `commonName` set, `subject=` comes back **blank**. That's expected — TLS clients validate hostnames against the **Subject Alternative Name** list, not the legacy CN field.

Because it's self-signed, **issuer == subject** (both empty) — the cert vouches for itself. That's why every client needs `-k` / `--insecure` against it.

```
$ ... | openssl x509 -noout -subject -text
subject=
X509v3 Subject Alternative Name:
    DNS:training-tls.local
```

GOTCHAS — THE ONES THAT BITE

Where people trip

- **secretName mismatch.** The Certificate writes to one Secret, the Ingress `tls:` references another → handshake fails or falls back to a default cert.
- **No ClusterIssuer.** The Certificate stays `READY: False` forever — `describe certificate` shows why.
- **Forgetting `-k`.** curl and browsers reject a self-signed cert by default — that's correct behaviour, not a bug.
- **`ssl-redirect` without `tls:`.** The annotation forces HTTPS, but with no TLS block there's nothing to redirect to.

YOUR TLS INGRESS TOOLKIT

Commands to keep at hand

```
$ kubectl get certificate -n NS                       # READY column
$ kubectl describe certificate NAME -n NS              # why it isn't Ready
$ kubectl get secret NAME -n NS -o jsonpath='{.type}'  # kubernetes.io/tls ?
$ kubectl describe ingress NAME -n NS                  # TLS section: secret + host
$ curl -k --resolve HOST:PORT:IP https://HOST:PORT     # -k for self-signed
```

Tip: `kubectl wait --for=condition=Ready certificate/...` beats polling — issuance is asynchronous.

RECAP

TLS Ingress in one breath

- ClusterIssuer → Certificate → Secret → Ingress `tls:`
- A Certificate is a request; cert-manager reconciles it asynchronously
- No `commonName` → empty Subject; trust rides on `dnsNames` / SAN
- Self-signed always needs `-k` on the client

HANDS-ON

Now run `Lab 4.6`

`labs/4.6/`

Next: **4.7 — Network Policies**